

GO MEETUP AMSTERDAM · August 26, 2026

The Curious Case of the *Missing Memory*

A 3 MiB heap, a 400 MiB alert, and an hour I'd like back.

Mohammed Nafees

hi, I'm Mohammed Nafees

Founding Engineer @ Hatchet (hatchet.run)

writing Go since 2021

CLIs, k8s, o11y, distributed systems, postgres

github.com/mnafees

3 a.m.

My phone rings. The ringtone is best described as “European ambulance”.

The alert

“High memory usage of controller pods”

Repeated and critical. According to kube-state-metrics and kubectl top, we were on the brink of OOMing.

First stop: pprof

Normally a few minutes of digging finds a memory spike. This time the profile told me nothing.

heap

3 MiB

what Kubernetes showed

400+ MiB

What the controller does

It writes batches of data to temporary files, then offloads them to S3 for cold storage.

the hot path

```
os.CreateTemp(...) → write batch → upload to S3 → os.Remove(...)
```

The alloc_space profile pointed here — nearly a third of every byte allocated since start.

Meanwhile, in production

The pod climbed toward its limit again. More Slack alerts.

They kept resolving on their own once the vertical pod autoscaler kicked in.

The process inside looked completely fine the whole time.

I had no intention of taking apart a running production pod at 3 AM.

A tiny program that does the same thing

Create a temp file, write a payload, remove it. Over and over.

```
the hot loop
f, _ := os.CreateTemp(dir, "batch-*")
f.Write(payload)
f.Close()
os.Remove(f.Name())
```

8 goroutines · 4 KiB payloads · distroless container · zero files on disk at steady state

kind cluster on my MacBook, 512 MiB limit

```
$ kubectl top pod
```

```
MEMORY (bytes)    409Mi    ← climbing toward the limit
```

Same behaviour as production, reproduced on a laptop. Which was a relief, because now I could take it apart.

But inside the container, everything is fine

I used `runtime.ReadMemStats` for the Go heap, and process RSS from `/proc/self/status`.

Go heap	2–3 MiB
---------	---------

process RSS	~15 MiB
-------------	---------

kubectl top	409 Mi
-------------	--------

...while doing over 100,000 operations per second.

The Go runtime couldn't help me any further.

So it was time to go and ask the Linux kernel instead.

cgroups, in one sentence

Control groups are how Linux groups processes so their resource usage can be limited and measured.

Kubernetes depends on them to enforce a pod's memory limit. When you write `limits.memory` in your YAML, a cgroup is what makes that real.

The good news: it's just text files

The container's memory cgroup exposes plain text files under `/sys/fs/cgroup`. No special mounts, no privileges. `cat` works.

```
check your cgroup version first  
$ stat -fc %T /sys/fs/cgroup  
cgroup2fs          ← cgroup v2
```

Our nodes run v2. The field names differ a bit on v1.

So I added a second reader

Right next to the ReadMemStats call: read the cgroup's own memory files and print both, side by side.

```
two views, one process  
runtime.ReadMemStats(&m)           // what Go thinks  
readCgroup("/sys/fs/cgroup/memory.stat") // what the kernel thinks
```

That's pretty much the entire trick.

There it is

CGROUP

```
anon          = 11.3Mi      ← the actual running Go program
slab_recl     = 441.7Mi    ← ???
```

Eleven megabytes of my program. Four hundred and forty of something else.

The slab

The allocator the Linux kernel uses for its own bookkeeping objects. Two of them matter here.

dentries directory entry cache — every path lookup and file creation adds one, so the kernel doesn't have to walk the filesystem again

inodes the in-memory representation of a file's metadata

What those two calls actually do

`os.CreateTemp`

creates a new dentry and a new inode

`os.Remove`

unlinks the file — but the kernel doesn't immediately free those entries

They go onto a reclaimable slab list and sit there as warm cache — ready for the next time you ask for that file.

100,000+ files per second.

We were creating dentries and inodes faster than the kernel reclaimed them. The slab kept growing until memory pressure forced it back down.

Confirming it

```
dropping the caches on the node  
$ echo 2 > /proc/sys/vm/drop_caches  
slab_recl → fell instantly
```

None of this was the process's memory. The kernel allocated it on the slab for its own caches.

The only way my program found it was by reading the cgroup files directly.

Why is that charged to us at all?

Kernel memory allocated on behalf of a cgroup is billed to that cgroup.

Which seems unfair, until you consider the alternative: a container could exhaust a node's kernel memory for free.

What was actually happening

Zero restarts.

Zero OOM kills.

The container was completely healthy.

kubectl top said we were nearing the limit — so our alerts fired.

A phantom memory tax.

It's not just us

`ipmi_exporter` #329

Each scrape wrote a unique temp file. At high target counts the dentry and inode slab grew, showed up as slab_reclaimable, while the exporter's own memory stayed flat — eventually an OOM.

`Docker forums`

Same symptoms, lots of dentry and inode slab. Nobody ever replied.

Why kubectl top counts it

kubectl top reads `container_memory_working_set_bytes` from the metrics API. The calculation is one subtraction:

$$\text{working_set} = \text{memory.current} - \text{inactive_file}$$

Everything else counts — including the slab that held our dentries and inodes.

Kubernetes uses this value for eviction, and to decide when to OOM containers.

Open since March 2017

“kubelet counts active page cache against memory.available (maybe it shouldn't?)”

kubernetes/kubernetes #43916 · 140+ comments · still open nine years later

The title is itself a question. I read the comments to understand why it's still open.

Two reasons it's still open

It's deliberate

cAdvisor's authors chose working set as total usage minus reclaimable file cache. Nothing else can be guaranteed to be reclaimed in time. And changing the definition would change behaviour for every workload out there.

Nobody agrees what's reclaimable

The argument is over active_file page cache. It can't be reclaimed directly — it goes through a multi-step deactivation. Counting it may cause premature evictions. Not counting it risks under-provisioning and real OOMs.

What we did about it

```
the alert  
- container_memory_working_set_bytes  
+ container_memory_rss
```

...and went back to sleep.

Memory usage isn't one number.

Go's runtime, the process's RSS, and the cgroup's working set answer three different questions. Any one of them can look alarming while the other two look fine.

When they disagree, read `/sys/fs/cgroup/memory.stat` before trusting the alert.

thanks

questions?